

C++ EQUALIZATOR PLUG-IN: DOCUMENTATION (ESP)

Aitor Pitarch Fontcuberta

Este documento tiene como objetivo principal servir como un registro técnico detallado que registra el proceso de diseño, arquitectura y desarrollo de un ecualizador paramétrico estéreo profesional, creado con el lenguaje C++ y el framework JUCE. El texto busca documentar cómo se han resuelto los desafíos técnicos propios de la programación de audio en tiempo real, asegurando una comunicación fluida y segura entre el motor de procesamiento y la interfaz de usuario.

En cuanto a su contenido, el documento describe la implementación de una arquitectura basada en el sistema de gestión de estado de JUCE (APVTS) para centralizar el control de parámetros como frecuencias, ganancias y factores de calidad. Explica detalladamente la configuración de una cadena de procesamiento DSP (Digital Signal Processing) que utiliza filtros IIR y diseños de Butterworth para lograr cortes de precisión de hasta 48 dB/Oct en canales independientes para estéreo. Además, aborda técnicas de optimización mediante el uso de plantillas de C++ para evitar la duplicidad de código y la gestión eficiente de la CPU mediante el apagado dinámico de nodos no utilizados.

Finalmente, el escrito detalla la creación de la infraestructura visual del plugin, incluyendo el diseño de controles personalizados y el desarrollo de un analizador de espectro basado en la Transformada Rápida de Fourier (FFT). También se documenta la implementación de una interfaz interactiva que permite al usuario visualizar la curva de respuesta de frecuencia y manipular los parámetros del audio directamente con el ratón sobre el gráfico, integrando así la retroalimentación visual y auditiva en una herramienta de grado profesional.

1. Definición del Layout de Parámetros (APVTS)

Se ha implementado la arquitectura base de los controles del plugin, utilizando el sistema de gestión de estado de JUCE para centralizar la comunicación entre el procesador y la interfaz de usuario.

```
EQ_SharedCode EQAudioProcessor

186 juce::AudioProcessorValueTreeState::ParameterLayout EQAudioProcessor::createParameterLayout()
187 {
188     juce::AudioProcessorValueTreeState::ParameterLayout layout;
189
190     layout.add(std::make_unique<juce::AudioParameterFloat>("LowCut Frequency",
191         "Low Cut Frequency",
192         juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
193         20.0f));
194
195     layout.add(std::make_unique<juce::AudioParameterFloat>("HighCut Frequency",
196         "HighCut Frequency",
197         juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
198         20000.0f));
199
200     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Frequency",
201         "Peak Frequency",
202         juce::NormalisableRange<float>(20.0f, 20000.0f, 1.0f, 1.0f),
203         750.0f));
204
205     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Gain",
206         "Peak Gain",
207         juce::NormalisableRange<float>(-24.0f, 24.0f, 0.5f, 1.0f),
208         0.0f));
209
210     layout.add(std::make_unique<juce::AudioParameterFloat>("Peak Quality",
211         "Peak Quality",
212         juce::NormalisableRange<float>(0.1f, 10.0f, 0.05f, 1.0f),
213         1.0f));
214
215     juce::StringArray stringArray;
216     for (int i = 0; i < 4; ++i)
217     {
218         juce::String str;
219         str << (12 + i * 12);
220         str << " dB/Oct";
221         stringArray.add(str);
222     }
223
224     layout.add(std::make_unique<juce::AudioParameterChoice>("LowCut Slope", "LowCut Slope", stringArray, 0));
225     layout.add(std::make_unique<juce::AudioParameterChoice>("HighCut Slope", "HighCut Slope", stringArray, 0));
226
227     return layout;
228 }
```

Parámetros y Lógica:

- Bandas de Frecuencia: Se definen tres puntos de control (LowCut, HighCut y Peak) con un rango de 20 Hz a 20 kHz. Se utiliza una resolución de 1.0 Hz.
- Control de Ganancia (Peak): Rango de ± 24 dB con pasos de 0.5 dB, permitiendo un control detallado sobre la amplitud de la banda paramétrica.
- Factor de Calidad (Q): Rango de 0.1 a 10.0 con pasos de 0.05, definiendo el ancho de banda del filtro de campana.
- Pendientes de Corte (Slopes): Implementación de una lógica de selección discreta mediante un bucle for que genera opciones de 12, 24, 36 y 48 dB/Oct para los filtros de paso alto y paso bajo.

Funciones y Variables Usadas:

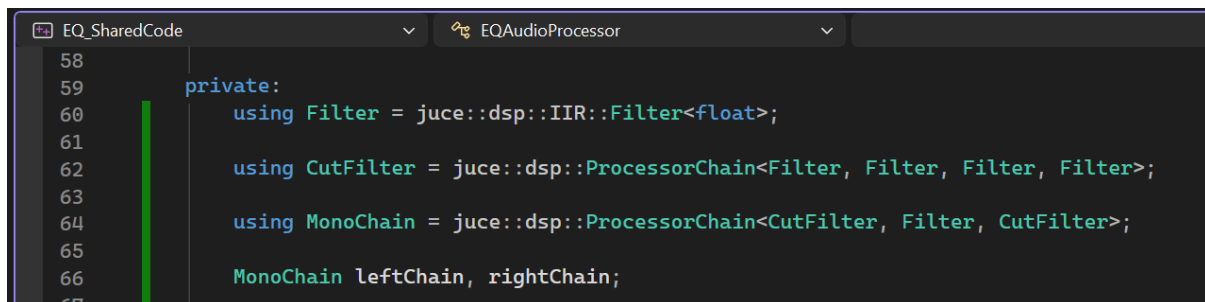
- *createParameterLayout()*: Método encargado de instanciar y retornar el objeto *ParameterLayout* necesario para la inicialización del *AudioProcessorValueTreeState*.
- *juce::AudioParameterFloat*: Clase para parámetros de valor continuo.
 - *parameterID*: String identificador único para automatización.
 - *parameterName*: Nombre legible por el usuario en el DAW.
 - *NormalisableRange<float>*: Objeto que define el comportamiento del slider (mín, máx, intervalo, skew).

- `defaultValue`: Valor inicial asignado al cargar el plugin.
- `juce::AudioParameterChoice`: Clase para parámetros de selección múltiple (menús desplegables).
 - `choices`: `StringArray` que contiene las etiquetas de texto de las opciones disponibles.
 - `defaultIndex`: Índice de la opción seleccionada por defecto.
- `juce::NormalisableRange<float>`:
 - `rangeStart / rangeEnd`: Límites del parámetro.
 - `intervalValue`: Incremento mínimo del valor.
 - `skewFactor`: Factor de asimetría (1.0 para respuesta lineal)

La estructura del APVTS permite una gestión de parámetros thread-safe, esencial para la sincronización entre el hilo de audio y el de la interfaz (GUI). El uso de `NormalisableRange` con pasos específicos garantiza que la cuantización de los valores de control sea coherente con los requisitos de precisión de un procesador de señal profesional.

2. Configuración de la Cadena de Procesado (DSP Chain)

Se ha definido la infraestructura de procesamiento de señal utilizando el módulo DSP de JUCE, estructurando los filtros en cadenas de procesamiento mono para gestionar el audio en estéreo de forma independiente.



```

58
59 private:
60     using Filter = juce::dsp::IIR::Filter<float>;
61
62     using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;
63
64     using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, CutFilter>;
65
66     MonoChain leftChain, rightChain;
67

```

Parámetros y Lógica:

- Jerarquía de Filtros: Se utiliza un alias `Filter` para filtros IIR de precisión simple.
- Estructura de Corte (`CutFilter`): Se define un `ProcessorChain` compuesto por 4 filtros en serie. Esta arquitectura permite sumar pendientes de 12 dB/Oct por cada filtro, logrando los cortes de hasta 48 dB/Oct definidos en el APVTS.
- Cadena Mono (`MonoChain`): Estructura final que organiza el flujo de señal en tres etapas: LowCut (`CutFilter`), Peak (`Filter`) y HighCut (`CutFilter`).
- Instanciación Estéreo: Se declaran dos instancias de `MonoChain` (`leftChain` y `rightChain`) para procesar los canales izquierdo y derecho por separado.

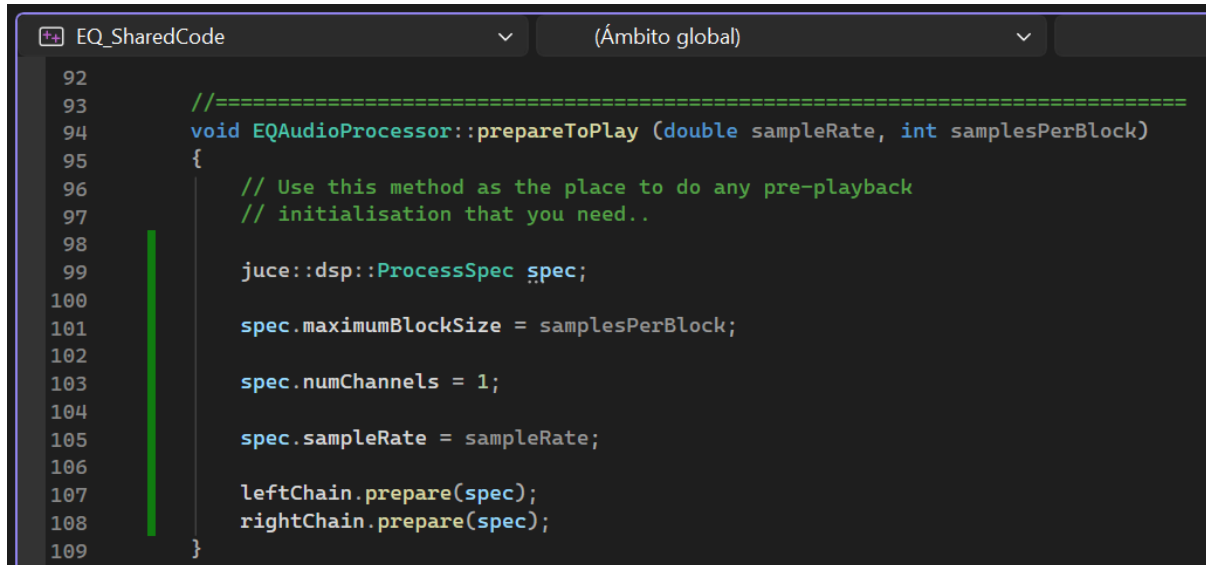
Funciones y Variables Usadas:

- `juce::dsp::IIR::Filter<float>`: Clase base para filtros de respuesta al impulso infinita.
- `juce::dsp::ProcessorChain`: Plantilla de clase que permite encadenar múltiples procesadores DSP, gestionando automáticamente el paso de muestras entre ellos.
- `using (Type Aliasing)`: Utilizado para simplificar la declaración de tipos complejos de plantillas (templates), mejorando la legibilidad del código.

La implementación establece una topología de filtros en serie. La modularidad de `ProcessorChain` permite tratar cada banda como una unidad lógica, facilitando la expansión de la pendiente del filtro de corte mediante la concatenación de instancias IIR de segundo orden.

3. Inicialización en prepareToPlay

Se ha implementado la preparación del motor DSP, asegurando que todos los componentes de la cadena de audio conozcan las especificaciones del entorno antes de comenzar la reproducción.



```
92
93 //=====
94 void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
95 {
96     // Use this method as the place to do any pre-playback
97     // initialisation that you need..
98
99     juce::dsp::ProcessSpec spec;
100
101     spec.maximumBlockSize = samplesPerBlock;
102
103     spec.numChannels = 1;
104
105     spec.sampleRate = sampleRate;
106
107     leftChain.prepare(spec);
108     rightChain.prepare(spec);
109 }
```

Parámetros y Lógica:

- Configuración del DSP: Se utiliza un objeto ProcessSpec para pasar la frecuencia de muestreo y el tamaño máximo de bloque a las cadenas de filtrado.
- Sincronización de Canales: Cada MonoChain se inicializa con un solo canal (numChannels = 1), ya que el procesamiento estéreo se divide en dos cadenas mono independientes.

Funciones y Variables Usadas:

- *prepareToPlay()*: Método del AudioProcessor que el DAW llama antes de empezar el procesamiento de audio.
- *juce::dsp::ProcessSpec*: Estructura que contiene:
 - *sampleRate*: Frecuencia de muestreo actual (p. ej. 44100 Hz).
 - *maximumBlockSize*: Tamaño máximo de los bloques de audio que procesará el plugin.
 - *numChannels*: Número de canales que manejará ese procesador específico.
- *prepare()*: Método que reserva memoria y precalcula coeficientes necesarios en cada elemento de la cadena DSP.

Antes de que empiece a sonar la música, el programa necesita saber a qué velocidad va el audio (sample rate) y cuánta información le llegará de golpe. Esta función le da esas instrucciones al ecualizador para que esté listo y no haya errores o ruidos extraños al dar al "play".

4. Implementación del Bucle de Procesado (processBlock)

Se ha configurado el núcleo del procesamiento en tiempo real, donde el audio entrante se descompone en bloques mono para ser filtrado por las cadenas DSP correspondientes.

```
EQ_SharedCode
EQAudioProcessor
hasEditor() const

143 void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
144 {
145     juce::ScopedNoDenormals noDenormals;
146     auto totalNumInputChannels = getTotalNumInputChannels();
147     auto totalNumOutputChannels = getTotalNumOutputChannels();
148
149     // In case we have more outputs than inputs, this code clears any output
150     // channels that didn't contain input data, (because these aren't
151     // guaranteed to be empty - they may contain garbage).
152     // This is here to avoid people getting screaming feedback
153     // when they first compile a plugin, but obviously you don't need to keep
154     // for this code if your algorithm always overwrites all the output channels.
155     for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
156         buffer.clear (i, 0, buffer.getNumSamples());
157
158     juce::dsp::AudioBlock<float> block(buffer);
159
160     auto leftBlock = block.getSingleChannelBlock(0);
161     auto rightBlock = block.getSingleChannelBlock(1);
162
163     juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
164     juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);
165
166     leftChain.process(leftContext);
167     rightChain.process(rightContext);
168 }
```

Parámetros y Lógica:

- Gestión de Denormales: Se utiliza ScopedNoDenormals para evitar caídas de rendimiento en la CPU causadas por números denormales (underflow) en los cálculos recursivos de los filtros IIR.
- Abstracción de Bloques: Se convierte el AudioBuffer nativo en un AudioBlock, facilitando la compatibilidad con el módulo DSP de JUCE.
- Procesado Contextual: Se crean contextos de proceso para cada canal, permitiendo que los filtros modifiquen el audio directamente "in-place".

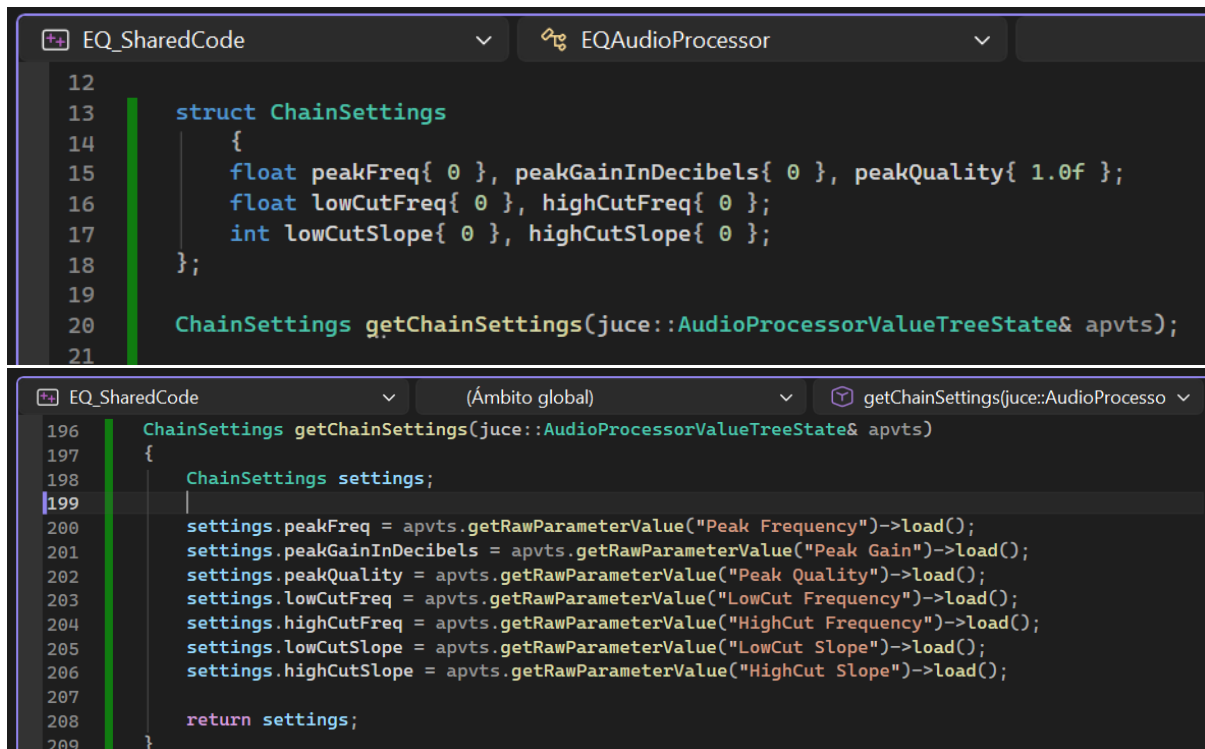
Funciones y Variables Usadas:

- *juce::ScopedNoDenormals*: Clase que optimiza el uso de la CPU durante el procesamiento evitando el manejo de números extremadamente cercanos a cero.
- *juce::dsp::AudioBlock<float>*: Clase que envuelve los datos del buffer de audio.
 - *getSingleChannelBlock(index)*: Extrae un puntero y metadatos de un canal específico del bloque.
- *juce::dsp::ProcessContextReplacing<float>*: Contexto que gestiona la entrada y salida de datos sobre la misma dirección de memoria, optimizando el acceso a caché.

El bloque de proceso segmenta el flujo de datos estéreo en flujos mono independientes. El uso de contextos de reemplazo (Replacing) asegura una eficiencia máxima al procesar los coeficientes del filtro directamente sobre el buffer de entrada, minimizando la latencia y el overhead de memoria.

5. Estructura de Datos y Extracción de Parámetros

Se ha definido una estructura auxiliar y una función de utilidad para centralizar la obtención de valores desde el `AudioProcessorValueTreeState`, facilitando su uso en el bucle de audio.



```
EQ_SharedCode EQAudioProcessor
12
13 struct ChainSettings
14 {
15     float peakFreq{ 0 }, peakGainInDecibels{ 0 }, peakQuality{ 1.0f };
16     float lowCutFreq{ 0 }, highCutFreq{ 0 };
17     int lowCutSlope{ 0 }, highCutSlope{ 0 };
18 };
19
20 ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts);
21

(Ámbito global) getChainSettings(juce::AudioProcesso
196 ChainSettings getChainSettings(juce::AudioProcessorValueTreeState& apvts)
197 {
198     ChainSettings settings;
199
200     settings.peakFreq = apvts.getRawParameterValue("Peak Frequency")->load();
201     settings.peakGainInDecibels = apvts.getRawParameterValue("Peak Gain")->load();
202     settings.peakQuality = apvts.getRawParameterValue("Peak Quality")->load();
203     settings.lowCutFreq = apvts.getRawParameterValue("LowCut Frequency")->load();
204     settings.highCutFreq = apvts.getRawParameterValue("HighCut Frequency")->load();
205     settings.lowCutSlope = apvts.getRawParameterValue("LowCut Slope")->load();
206     settings.highCutSlope = apvts.getRawParameterValue("HighCut Slope")->load();
207
208     return settings;
209 }
```

Parámetros y Lógica:

- Estructura `ChainSettings`: Agrupa todas las variables de control (frecuencias, ganancias, calidades y pendientes) en un solo objeto para evitar llamadas repetitivas y dispersas al APVTS.
- Función `getChainSettings`: Realiza la lectura de los valores actuales de los parámetros. Utiliza `load()` para garantizar una lectura atómica y segura desde el hilo de audio.
- Mapeo de Tipos: Las frecuencias y ganancias se extraen como `float`, mientras que los índices de las pendientes (slopes) se recuperan como `int`.

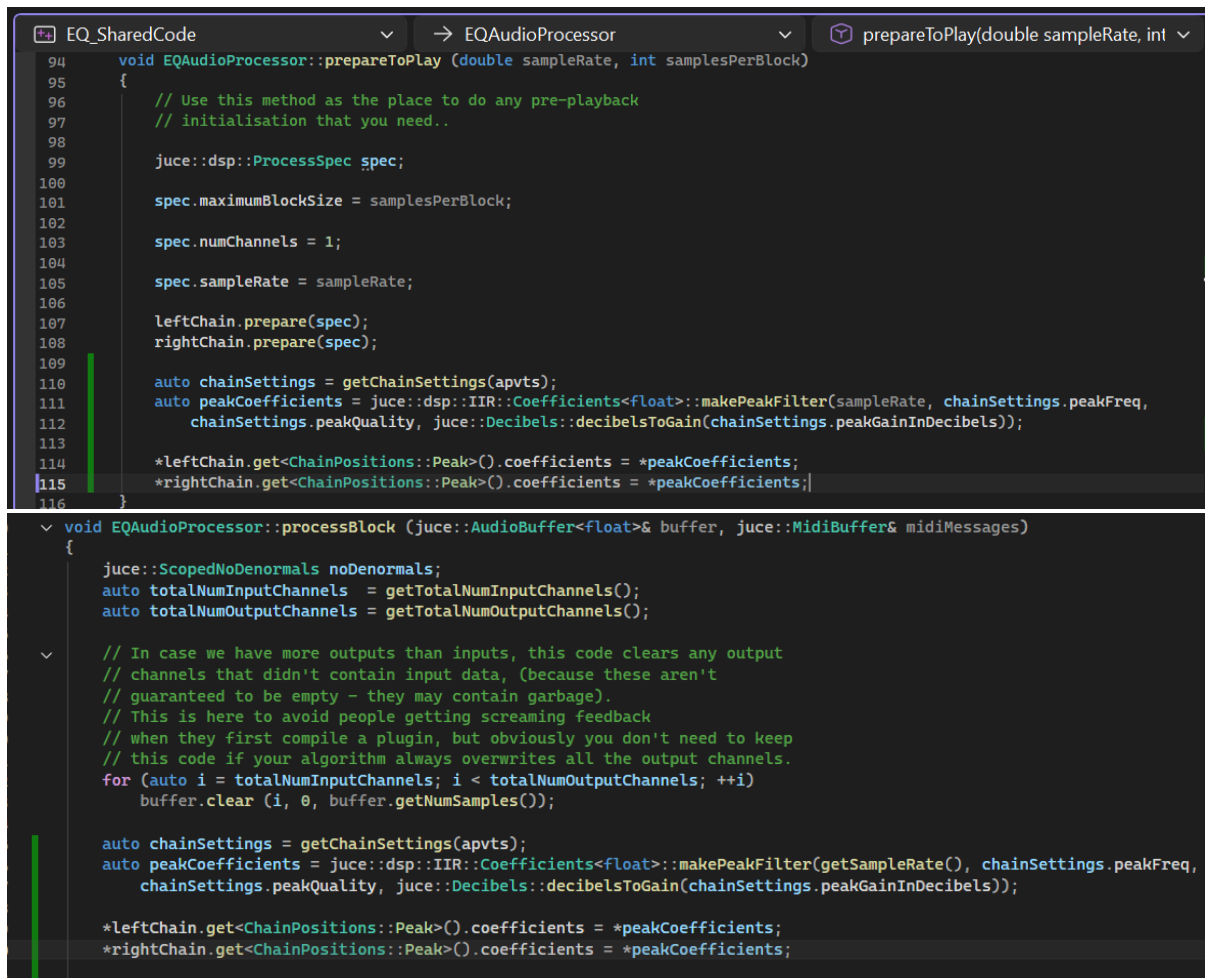
Funciones y Variables Usadas:

- `apvts.getRawParameterValue()`: Obtiene un puntero al valor en tiempo real de un parámetro mediante su ID.
- `load()`: Método de `std::atomic` que asegura que el valor leído sea el más reciente y consistente entre diferentes hilos de ejecución.

La implementación de una función de extracción dedicada desacopla la lógica de obtención de datos de la lógica de procesamiento. Esto mejora la mantenibilidad y asegura que el hilo de audio trabaje con una "instantánea" coherente de todos los controles al inicio de cada bloque.

6. Actualización de Coeficientes en Tiempo Real

Se ha integrado la lógica necesaria para transformar los parámetros de control en coeficientes matemáticos aplicables al filtro Peak, tanto en la inicialización como en el proceso continuo.



```

94 void EQAudioProcessor::prepareToPlay (double sampleRate, int samplesPerBlock)
95 {
96     // Use this method as the place to do any pre-playback
97     // initialisation that you need..
98
99     juce::dsp::ProcessSpec spec;
100
101     spec.maximumBlockSize = samplesPerBlock;
102
103     spec.numChannels = 1;
104
105     spec.sampleRate = sampleRate;
106
107     leftChain.prepare(spec);
108     rightChain.prepare(spec);
109
110     auto chainSettings = getChainSettings(apvts);
111     auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
112     chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));
113
114     *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
115     *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
116 }
117
118 void EQAudioProcessor::processBlock (juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
119 {
120     juce::ScopedNoDenormals noDenormals;
121     auto totalNumInputChannels = getTotalNumInputChannels();
122     auto totalNumOutputChannels = getTotalNumOutputChannels();
123
124     // In case we have more outputs than inputs, this code clears any output
125     // channels that didn't contain input data, (because these aren't
126     // guaranteed to be empty - they may contain garbage).
127     // This is here to avoid people getting screaming feedback
128     // when they first compile a plugin, but obviously you don't need to keep
129     // this code if your algorithm always overwrites all the output channels.
130     for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
131         buffer.clear(i, 0, buffer.getNumSamples());
132
133     auto chainSettings = getChainSettings(apvts);
134     auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(getSampleRate(), chainSettings.peakFreq,
135     chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));
136
137     *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
138     *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
139 }

```

Parámetros y Lógica:

- Cálculo de Coeficientes: Se utiliza la función estática makePeakFilter para generar los coeficientes de un filtro de campana basados en la frecuencia central, el factor Q y la ganancia.
- Conversión de Unidades: Se transforma la ganancia de decibelios (dB) a ganancia lineal mediante decibelsToGain, requisito necesario para el motor DSP.
- Asignación a la Cadena: Se accede al elemento específico de la cadena de procesamiento (ChainPositions::Peak) y se actualizan sus coeficientes de forma sincronizada para los canales izquierdo y derecho.

Funciones y Variables Usadas:

- *juce::dsp::IIR::Coefficients<float>::makePeakFilter*: Calcula los coeficientes del filtro IIR basándose en:
 - sampleRate: Frecuencia de muestreo.
 - frequency: Frecuencia central de la campana.
 - Q: Factor de calidad (ancho de banda).
 - gain: Ganancia lineal.

- *leftChain.get<Index>()*: Función de plantilla que permite acceder a un procesador específico dentro de una ProcessorChain en tiempo de compilación.
- **coefficients = *newCoefficients*: Operación de desreferenciación para copiar los valores de los nuevos coeficientes en el puntero gestionado por el filtro.

La actualización de coeficientes en el processBlock permite que el filtro responda instantáneamente a los cambios en la interfaz. Al replicar los mismos coeficientes en leftChain y rightChain, se garantiza una respuesta de fase y magnitud idéntica en ambos canales, manteniendo la coherencia de la imagen estéreo.

7. Integración y Lógica del Filtro LowCut (High-Pass)

Se ha implementado el procesamiento del filtro de corte de graves (LowCut), utilizando una topología de filtro Butterworth de orden variable. La pendiente del filtro (slope) determina cuántos eslabones de la cadena de procesamiento se activan para alcanzar la atenuación deseada.

```

103     auto chainSettings = getChainSettings(apvts);
104     auto peakCoefficients = juce::dsp::IIR::Coefficients<float>::makePeakFilter(sampleRate, chainSettings.peakFreq,
105                                     chainSettings.peakQuality, juce::Decibels::decibelsToGain(chainSettings.peakGainInDecibels));
106
107     auto cutCoefficients = juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
108                                                         getSampleRate(),
109                                                         2 * (chainSettings.lowCutSlope + 1));
110
111     *leftChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
112     *rightChain.get<ChainPositions::Peak>().coefficients = *peakCoefficients;
113
114     // --- CONFIGURACIÓN LOW CUT IZQUIERDO ---
115     auto& leftLowCut = leftChain.get<ChainPositions::LowCut>();
116     leftLowCut.setBypassed<0>(true);
117     leftLowCut.setBypassed<1>(true);
118     leftLowCut.setBypassed<2>(true);
119     leftLowCut.setBypassed<3>(true);
120

```

```

switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *leftLowCut.get<0>().coefficients = *cutCoefficients[0];
    leftLowCut.setBypassed<0>(false);
    *leftLowCut.get<1>().coefficients = *cutCoefficients[1];
    leftLowCut.setBypassed<1>(false);
    *leftLowCut.get<2>().coefficients = *cutCoefficients[2];
    leftLowCut.setBypassed<2>(false);
    *leftLowCut.get<3>().coefficients = *cutCoefficients[3];
    leftLowCut.setBypassed<3>(false);
    break;
}
}

```

```

switch (chainSettings.lowCutSlope)
{
case Slope::Slope_12:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    break;
}
case Slope::Slope_24:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    break;
}
case Slope::Slope_36:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    break;
}
case Slope::Slope_48:
{
    *rightLowCut.get<0>().coefficients = *cutCoefficients[0];
    rightLowCut.setBypassed<0>(false);
    *rightLowCut.get<1>().coefficients = *cutCoefficients[1];
    rightLowCut.setBypassed<1>(false);
    *rightLowCut.get<2>().coefficients = *cutCoefficients[2];
    rightLowCut.setBypassed<2>(false);
    *rightLowCut.get<3>().coefficients = *cutCoefficients[3];
    rightLowCut.setBypassed<3>(false);
    break;
}
}

```

Parámetros y Lógica:

- Diseño del Filtro Butterworth: Se generan los coeficientes para un filtro paso-alto (High-Pass) basándose en la frecuencia de corte (lowCutFreq) extraída del APVTS.
- Cálculo del Orden del Filtro: La función requiere el orden del filtro matemático. Como cada opción de nuestro menú (Slope_12, Slope_24, etc.) representa un índice (0, 1, 2, 3), la fórmula $2 * (\text{chainSettings.lowCutSlope} + 1)$ traduce este índice en los órdenes correctos (2, 4, 6, 8 correspondientemente).
- Gestión de la Cadena (Bypass): Nuestro CutFilter está compuesto por hasta 4 filtros individuales (cada uno aportando 12 dB/Oct). Por defecto, se ponen todos en estado de bypass (apagados) al inicio del bloque.

Funciones y Variables Usadas:

- `juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod`: Retorna un `ReferenceCountedArray` que contiene los coeficientes para los distintos filtros IIR en cascada necesarios para formar el filtro de orden superior.
- `setBypassed<Index>(bool)`: Método de `juce::dsp::ProcessorChain` que activa (false) o desactiva (true) un procesador específico dentro de la cadena sin sacarlo del flujo de la memoria, ahorrando CPU cuando no se necesitan pendientes pronunciadas.
- Desreferenciación de Coeficientes (`*cutCoefficients[i]`): Extrae el bloque de coeficientes específico del array generado para asignarlo a la posición *i* de nuestra subcadena `LowCut`.

La implementación de pendientes variables se logra mediante la cascada de múltiples filtros IIR de segundo orden (bicuadráticos). Al utilizar la aproximación de Butterworth, garantizamos una respuesta en frecuencia lo más plana posible en la banda de paso (sin rizado) a expensas de una transición de fase no lineal. El apagado dinámico de los nodos no utilizados mediante `setBypassed` optimiza los ciclos de CPU, asegurando que el motor de audio solo procese las operaciones matemáticas estrictamente necesarias para la pendiente seleccionada (ej. 1 filtro para 12 dB/Oct, 4 filtros para 48 dB/Oct).

8. Refactorización y Optimización del Código

A medida que el proyecto escaló para soportar procesamiento estéreo (canales izquierdo y derecho) y se preparó el terreno para añadir más filtros (como el HighCut), el código en `prepareToPlay` y `processBlock` presentaba una alta duplicidad. Aplicando el principio DRY (Don't Repeat Yourself), se ha refactorizado la lógica de actualización de los filtros mediante el uso de plantillas (templates) de C++ en el archivo de cabecera (`PluginProcessor.h`).

```
using Coefficients = Filter::CoefficientsPtr;
static void updateCoefficients(Coefficients& old, const Coefficients& replacements);

template<int Index, typename ChainType, typename CoefficientType>
void update(ChainType& chain, const CoefficientType& coefficients)
{
    updateCoefficients(chain.template get<Index>().coefficients, coefficients[Index]);
    chain.template setBypassed<Index>(false);
}
```

```
template<typename ChainType, typename CoefficientType>
void updateCutFilter(ChainType& chain,
                    const CoefficientType& coefficients,
                    const Slope& slope)
{
    chain.template setBypassed<0>(true);
    chain.template setBypassed<1>(true);
    chain.template setBypassed<2>(true);
    chain.template setBypassed<3>(true);

    switch (slope)
    {
        case Slope_48:
        {
            update<3>(chain, coefficients);
        }
        case Slope_36:
        {
            update<2>(chain, coefficients);
        }
        case Slope_24:
        {
            update<1>(chain, coefficients);
        }
        case Slope_12:
        {
            update<0>(chain, coefficients);
        }
    }
}
```

Nuevas Funciones de Ayuda (Helper Functions):

- *updateCoefficients*: Una función estática encargada de intercambiar los coeficientes antiguos por los nuevos de manera segura en la memoria.
- *update<Index, ChainType, CoefficientType>*: Una plantilla que abstrae la actualización de un eslabón individual de la cadena. Recibe el índice del filtro, asigna los coeficientes correspondientes y lo activa (sacándolo del estado de bypass).
- *updateCutFilter*: El núcleo de la refactorización para los filtros de corte. Agrupa la lógica que antes requería bloques switch masivos y duplicados para cada canal.

A diferencia de un switch tradicional donde cada case termina con un break, aquí se han omitido deliberadamente. Esto significa que si el usuario selecciona una pendiente de 48 dB/Oct (`Slope_48`), el código ejecuta la actualización del cuarto filtro (`update<3>`), y "cae" en cascada ejecutando también los casos de 36, 24 y 12 dB/Oct.

9. Integración del Filtro HighCut y Finalización de la Cadena DSP

Una vez establecida la arquitectura óptima mediante el uso de plantillas (templates) para evitar la duplicación de código, se ha procedido a integrar el último eslabón de nuestra cadena de ecualización actual: el filtro HighCut (también conocido como filtro paso-bajo o Low-Pass).

```
void EQAudioProcessor::processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear(i, 0, buffer.getNumSamples());

    //Peak update-----
    auto chainSettings = getChainSettings(apvts);
    updatePeakFilter(chainSettings);
    //-----

    //LowCut update-----
    auto cutCoefficients = juce::dsp::FilterDesign<float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.lowCutSlope + 1));

    //LeftLowCut update
    auto& leftLowCut = leftChain.getChainPositions::LowCut();
    updateCutFilter(leftLowCut, cutCoefficients, chainSettings.lowCutSlope);

    //rightLowCut update
    auto& rightLowCut = rightChain.getChainPositions::LowCut();
    updateCutFilter(rightLowCut, cutCoefficients, chainSettings.lowCutSlope);
    //-----

    //HighCut update-----
    auto highCutCoefficients = juce::dsp::FilterDesign<float>::designIIRLowpassHighOrderButterworthMethod(chainSettings.highCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.highCutSlope + 1));

    //LeftHighCut
    auto& leftHighCut = leftChain.getChainPositions::HighCut();
    updateCutFilter(leftHighCut, highCutCoefficients, chainSettings.highCutSlope);

    //RightHighCut
    auto& rightHighCut = rightChain.getChainPositions::HighCut();
    updateCutFilter(rightHighCut, highCutCoefficients, chainSettings.highCutSlope);
    //-----

    //Final audio processing-----
    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBlock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

Evolución de prepareToPlay y processBlock:

1. Generación de Coeficientes: Ahora se calculan simultáneamente los coeficientes para las tres bandas (*peakCoefficients*, *lowCutCoefficients* y *highCutCoefficients*). Para el HighCut, se utiliza la función *designIIRLowpassHighOrderButterworthMethod* de JUCE, que, al igual que el LowCut, requiere la frecuencia de corte, la frecuencia de muestreo y el orden del filtro matemático calculado a partir de la pendiente (slope) seleccionada.
2. Actualización Simplificada de Nodos: En lugar de escribir múltiples bloques switch para evaluar qué filtros encender o apagar en cada canal, el flujo se ha reducido a llamadas directas y semánticas a nuestras funciones de ayuda:
 - Se actualizan los coeficientes del Peak filter de forma directa usando la función *updateCoefficients*.
 - Se llama a *updateCutFilter* pasándole la cadena correspondiente (leftChain o rightChain), los coeficientes generados y el slope seleccionado por el usuario en el APVTS. Esta función, por debajo, se encarga de todo el enrutamiento de los nodos activos y en bypass.
3. Procesado Estéreo Limpio: Finalmente, los bloques de audio de los canales izquierdo (0) y derecho (1) se extraen del buffer principal y se pasan a través de sus respectivas cadenas de procesamiento (leftChain.process y rightChain.process).

El método prepareToPlay ha sido actualizado exactamente con esta misma estructura. Esto garantiza que cuando el motor de audio arranque o cambie su configuración de hardware (como el sample rate), todos los filtros en todas las posiciones (LowCut, Peak, HighCut) estén inicializados con los coeficientes correctos antes de procesar el primer bloque de audio, previniendo chasquidos o comportamientos inestables.

10. Limpiando el processBlock con helper functions.

Aunque el uso de plantillas (templates) en el paso anterior mejoró mucho el ruteo de los filtros, el método `processBlock` seguía asumiendo la responsabilidad de calcular la matemática de los coeficientes de audio. Su única misión debería ser procesar audio lo más rápido posible. Por ello, hemos extraído toda la lógica de cálculo y actualización a funciones dedicadas y semánticas.

```
void EQAudioProcessor::updateLowCutFilters(const ChainSettings& chainSettings)
{
    auto cutCoefficients = juce::dsp::FilterDesign<Float>::designIIRHighpassHighOrderButterworthMethod(chainSettings.lowCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.lowCutSlope + 1));

    auto& leftLowCut = leftChain.getChainPositions::LowCut();
    auto& rightLowCut = rightChain.getChainPositions::LowCut();
    updateCutFilter(leftLowCut, cutCoefficients, chainSettings.lowCutSlope);
    updateCutFilter(rightLowCut, cutCoefficients, chainSettings.lowCutSlope);
}

void EQAudioProcessor::updateHighCutFilters(const ChainSettings& chainSettings)
{
    auto highCutCoefficients = juce::dsp::FilterDesign<Float>::designIIRLowpassHighOrderButterworthMethod(chainSettings.highCutFreq,
                                                                                                     getSampleRate(),
                                                                                                     2 * (chainSettings.highCutSlope + 1));

    auto& leftHighCut = leftChain.getChainPositions::HighCut();
    auto& rightHighCut = rightChain.getChainPositions::HighCut();
    updateCutFilter(leftHighCut, highCutCoefficients, chainSettings.highCutSlope);
    updateCutFilter(rightHighCut, highCutCoefficients, chainSettings.highCutSlope);
}

void EQAudioProcessor::updateFilters()
{
    auto chainSettings = getChainSettings(apvts);
    updatePeakFilter(chainSettings);
    updateLowCutFilters(chainSettings);
    updateHighCutFilters(chainSettings);
}
```

Nuevas Funciones de Control (Helper Methods):

Se han implementado métodos específicos para aislar la lógica de cada banda:

- *updatePeakFilter*: Obtiene los parámetros del usuario, calcula los coeficientes IIR para el filtro de campana y los aplica a ambos canales usando `updateCoefficients`.
- *updateLowCutFilters* y *updateHighCutFilters*: Encapsulan la llamada a la función de diseño de filtros Butterworth (calculando dinámicamente el orden según la pendiente) y pasan los resultados a nuestro template `updateCutFilter` para gestionar la cadena estéreo.
- *updateFilters*: Función que agrupa las llamadas a las funciones anteriores, asegurando que toda la EQ se actualice en un solo paso.

```
void EQAudioProcessor::processBlock(juce::AudioBuffer<float>& buffer, juce::MidiBuffer& midiMessages)
{
    juce::ScopedNoDenormals noDenormals;
    auto totalNumInputChannels = getTotalNumInputChannels();
    auto totalNumOutputChannels = getTotalNumOutputChannels();

    for (auto i = totalNumInputChannels; i < totalNumOutputChannels; ++i)
        buffer.clear(i, 0, buffer.getNumSamples());

    updateFilters();

    juce::dsp::AudioBlock<float> block(buffer);

    auto leftBlock = block.getSingleChannelBlock(0);
    auto rightBlock = block.getSingleChannelBlock(1);

    juce::dsp::ProcessContextReplacing<float> leftContext(leftBlock);
    juce::dsp::ProcessContextReplacing<float> rightContext(rightBlock);

    leftChain.process(leftContext);
    rightChain.process(rightContext);
}
```

Al extraer toda la lógica matemática a funciones de ayuda y agruparlas bajo `updateFilters()`, logramos un `processBlock` extremadamente limpio y enfocado exclusivamente en procesar el audio, que es su verdadera y única responsabilidad. Esta abstracción elimina por completo las duplicidades de código entre el bloque de procesamiento y `prepareToPlay`, facilitando enormemente la legibilidad del proyecto y dejándolo perfectamente preparado para escalar sin fricciones si en el futuro decidimos incorporar nuevas bandas de ecualización.

11. Infraestructura de Comunicación Segura: `SingleChannelSampleFifo`

Para poder visualizar el espectro de audio en la interfaz gráfica sin comprometer el rendimiento del procesamiento de la señal, se ha implementado un búfer circular de tipo FIFO (First-In, First-Out). Esta estructura permite extraer las muestras de audio desde el hilo de alta prioridad de manera completamente segura y sin bloqueos (lock-free).

```
template<typename BlockType>
struct SingleChannelSampleFifo
{
    SingleChannelSampleFifo(Channel ch) : channelToUse(ch)
    {
        prepared.set(false);
    }

    // Llamado desde prepareToPlay (audio-thread, antes de procesar)
    void prepare(int bufferSize)
    {
        prepared.set(false);
        size.set(bufferSize);

        bufferToFill.setSize(1, // 1 canal (mono)
                             bufferSize,
                             false, // keepExistingContent
                             true, // clearExtraSpace
                             true); // avoidReallocating

        abstractFifo.setTotalSize(bufferSize);
        prepared.set(true);
    }

    // Llamado desde processBlock - copia el canal indicado al FIFO
    void update(const BlockType& buffer)
    {
        jassert(prepared.get());
        jassert(buffer.getNumChannels() > channelToUse);

        auto* channelPtr = buffer.getReadPointer(channelToUse);

        int start1, size1, start2, size2;
        abstractFifo.prepareToWrite(buffer.getNumSamples(),
                                    start1, size1, start2, size2);

        if (size1 > 0)
            bufferToFill.copyFrom(0, start1, channelPtr, size1);
        if (size2 > 0)
            bufferToFill.copyFrom(0, start2, channelPtr + size1, size2);

        abstractFifo.finishedWrite(size1 + size2);
        numSamplesAvailable.set(abstractFifo.getNumReady());
    }
};
```

```
// Llamado desde la GUI - devuelve true si había suficientes muestras
bool getNextAudioBlock(BlockType& destination)
{
    if (!prepared.get() ||
        abstractFifo.getNumReady() < destination.getNumSamples())
        return false;

    int start1, size1, start2, size2;
    abstractFifo.prepareToRead(destination.getNumSamples(),
                                start1, size1, start2, size2);

    if (size1 > 0)
        destination.copyFrom(0, 0, bufferToFill, 0, start1, size1);
    if (size2 > 0)
        destination.copyFrom(0, size1, bufferToFill, 0, start2, size2);

    abstractFifo.finishedRead(size1 + size2);
    numSamplesAvailable.set(abstractFifo.getNumReady());
    return true;
}

int getNumSamplesAvailable() const { return numSamplesAvailable.get(); }
bool isPrepared() const { return prepared.get(); }
int getSize() const { return size.get(); }

private:
    Channel channelToUse;
    juce::AbstractFifo abstractFifo { 1 };
    BlockType bufferToFill;
    juce::Atomic<bool> prepared { false };
    juce::Atomic<int> size { 0 };
    juce::Atomic<int> numSamplesAvailable { 0 };
};
```

Flujo de Datos y Concurrencia:

- **Gestión Lock-Free:** Se utiliza la clase `juce::AbstractFifo` junto con un `std::array` estático para gestionar los índices de lectura y escritura. Esto asegura que el hilo de audio (Audio Thread) y el hilo de la interfaz (GUI Thread) nunca compitan por el mismo bloque de memoria de manera perjudicial.
- **Extracción en Tiempo Real:** En el método `processBlock` del procesador, una vez que el audio ha pasado por toda la cadena de filtros DSP, se envía una copia de las muestras procesadas de los canales izquierdo y derecho a sus respectivos FIFOs mediante los métodos `leftChannelFifo.push()` y `rightChannelFifo.push()`.
- **Desacoplamiento Estructural:** El procesador de audio ("Backend") se limita a "empujar" datos de forma continua, delegando por completo la responsabilidad de extraer y procesar esos datos al entorno gráfico.

El uso de una estructura Lock-Free es un principio de diseño crítico en el desarrollo de plugins de audio profesionales. Evita el uso de mutexes o candados de bloqueo que podrían provocar interrupciones (glitches) en el audio si el hilo de procesamiento tuviera que esperar al hilo de la interfaz gráfica para escribir nuevos datos en el búfer.

12. Motor del Analizador de Espectro (FFT)

Como base para la visualización del espectro de frecuencias del audio entrante/saliente, se ha creado la estructura lógica SpectrumAnalyzer. Este motor extrae los datos del FIFO mencionado anteriormente y los convierte del dominio del tiempo al dominio de la frecuencia.

```
SpectrumAnalyzer(SingleChannelSampleFifo<juce::AudioBuffer<float>>& fifo)
: fifo(fifo),
  forwardFFT(FFT_ORDER),
  window(FFT_SIZE + 1, juce::dsp::WindowingFunction<float>::hann)
{
    fftData.assign(2 * FFT_SIZE, 0.0f);
    monoBuffer.resize(FFT_SIZE, 0.0f);
    drawingData.fill(0.0f);
}
```

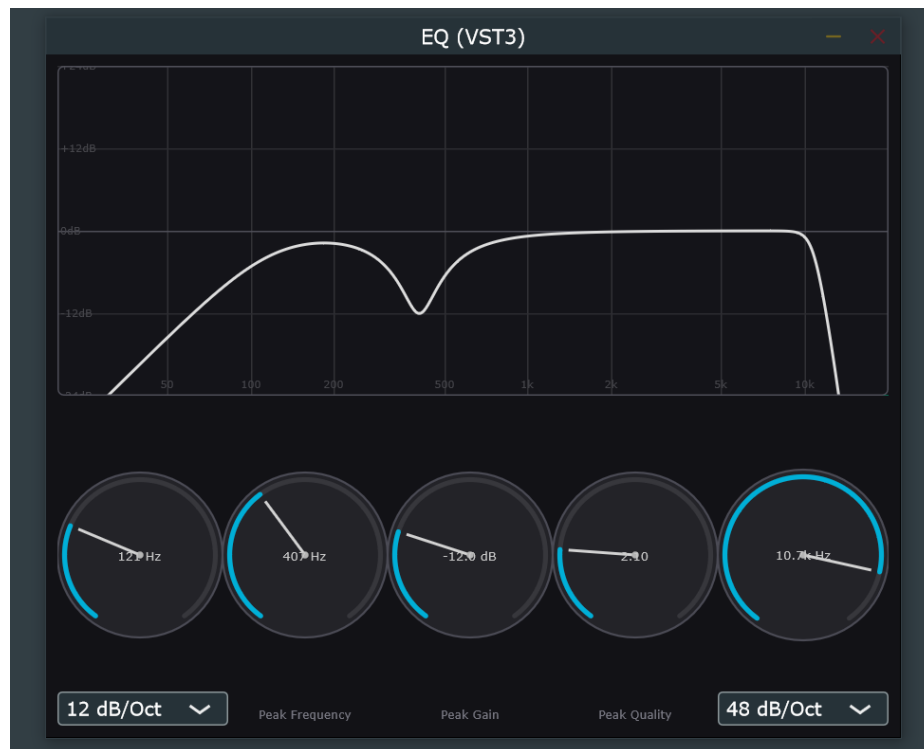
Procesamiento en el Hilo de la Interfaz:

- Transformada Rápida de Fourier (FFT): Se hace uso del módulo juce::dsp::FFT. Se ha configurado con un "Order" de 11, lo que determina un tamaño de ventana de 2048 muestras (2^{11}), ofreciendo una alta resolución de las frecuencias graves.
- Enventanado (Windowing): Antes de aplicar la FFT, los bloques de audio sin procesar pasan por una función de ventana tipo Hann (juce::dsp::WindowingFunction). Esto suaviza los bordes del bloque de muestras, evitando imperfecciones matemáticas (spectral leakage) que generarían artefactos visuales falsos.
- Generación de Bins: El método process() lee las muestras acumuladas, las transforma y calcula los valores absolutos resultantes, normalizándolos y dejándolos preparados en un array interno listos para ser dibujados en pantalla.

Desplazar los costosos cálculos matemáticos de la FFT al hilo de la interfaz (GUI Thread) asegura que la CPU dedique sus recursos críticos íntegramente al processBlock. Actualmente la FFT no se visualiza en el plug-in y queda pendiente de arreglar en una próxima actualización.

13. Visualización de la Curva de Respuesta

Se ha implementado un componente dedicado exclusivamente a trazar la línea que representa cómo están afectando los filtros a la señal a lo largo del espectro. Para ello, en lugar de medir el audio, se interroga matemáticamente a la cadena de filtros DSP.



Simulación del Motor DSP en la GUI

- Cadena de Ecualización Monocanal: El ResponseCurveComponent crea su propia MonoChain interna. Al inicializarse o al detectar cambios (mediante parameterChanged), actualiza esta cadena simulada con los mismos coeficientes del procesador real.
- Cálculo de Magnitudes Logarítmicas: Utilizando un juce::Timer que corre a 60 Hz, el componente itera a través del ancho en píxeles de la pantalla, calculando a qué frecuencia exacta corresponde cada píxel en una escala logarítmica (función mapFreqToX). Luego, pide al motor DSP la magnitud exacta de respuesta en esa frecuencia mediante getMagnitudeForFrequency.
- Generación de Gráficos Vectoriales: Se traza un juce::Path conectando las magnitudes de ganancia obtenidas para cada frecuencia, lo que dibuja una representación fidedigna y en tiempo real de los cortes y realces de ecualización.

El enfoque adoptado permite separar el estado visual del estado de procesamiento real. Al utilizar implementaciones de "Listeners" de parámetros (juce::AudioProcessorParameter::Listener), el componente se asegura de que la curva solo se redibuje y recalcula cuando el usuario interactúa realmente con la interfaz o automatiza los valores, garantizando un uso altamente eficiente de la CPU gráfica.

14. Interfaz Gráfica y Controles Personalizados

El diseño genérico por defecto ha sido reemplazado. Se han implementado controles físicos interactivos (sliders rotatorios y menús) que se enlazan de forma bidireccional con el AudioProcessorValueTreeState (APVTS).

```
void RotarySliderWithLabels::LookAndFeel::drawRotarySlider(
    juce::Graphics& g,
    int x, int y, int width, int height,
    float sliderPosProportional,
    float rotaryStartAngle,
    float rotaryEndAngle,
    juce::Slider& slider)
{
    using namespace juce;

    // Reservamos 16px abajo para el nombre del parámetro
    auto labelH = 16;
    auto bounds = Rectangle<float>(x, y, width, height - labelH);
    auto radius = jmin(bounds.getWidth(), bounds.getHeight()) / 2.0f;
    auto center = bounds.getCentre();

    // --- Fondo del knob ---
    g.setColour(Colour(35, 35, 40));
    g.fillEllipse(bounds.withSizeKeepingCentre(radius * 2.0f, radius * 2.0f));

    // --- Borde exterior ---
    g.setColour(Colour(75, 75, 85));
    g.drawEllipse(bounds.withSizeKeepingCentre(radius * 2.0f, radius * 2.0f), 1.5f);

    // --- Arco de recorrido total (gris oscuro) ---
    auto trackR = radius - 5.0f;
    Path arcBg;
    arcBg.addCentredArc(center.x, center.y, trackR, trackR,
        0.0f, rotaryStartAngle, rotaryEndAngle, true);
    g.setColour(Colour(55, 55, 60));
    g.strokePath(arcBg, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

    // --- Arco de valor (azul-cian) ---
    float angle = rotaryStartAngle + sliderPosProportional * (rotaryEndAngle - rotaryStartAngle);
    Path arcVal;
    arcVal.addCentredArc(center.x, center.y, trackR, trackR,
        0.0f, rotaryStartAngle, angle, true);
    g.setColour(Colour(0, 175, 215));
    g.strokePath(arcVal, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

    // --- Línea indicadora (desde centro hasta borde interior) ---
    auto thumbR = radius - 11.0f;
    Point<float> thumbPt(center.x + thumbR * std::sin(angle),
        center.y - thumbR * std::cos(angle));
    g.setColour(Colour(210, 210, 210));
    g.drawLine(Line<float>(center, thumbPt), 2.0f);

    // --- Punto central ---
    g.setColour(Colour(180, 180, 180));
    g.fillEllipse(Rectangle<float>(5.0f, 5.0f).withCentre(center));

    // --- Valor numérico (centrado en el knob) ---
    if (auto* rsw = dynamic_cast<RotarySliderWithLabels*>(&slider))
    {
        g.setColour(Colour(200, 200, 200));
        g.setFont(FontOptions(10.5f));
        g.drawFittedText(rsw->getDisplayString(),
            bounds.withSizeKeepingCentre(radius * 1.4f, radius * 0.6f).toNearestInt(),
            Justification::centred, 1);

        // --- Nombre del parámetro debajo del knob ---
        g.setColour(Colour(120, 120, 130));
        g.setFont(FontOptions(10.0f));
        auto labelBounds = Rectangle<int>(x, y + height - labelH, width, labelH);
        g.drawFittedText(rsw->param->getName(24), labelBounds, Justification::centred, 1);
    }
}
```

```

// --- Arco de valor (azul-cian) ---
float angle = rotaryStartAngle + sliderPosProportional * (rotaryEndAngle - rotaryStartAngle);
Path arcVal;
arcVal.addCentredArc(center.x, center.y, trackR, trackR,
    0.0f, rotaryStartAngle, angle, true);
g.setColour(Colour(0, 175, 215));
g.strokePath(arcVal, PathStrokeType(3.5f, PathStrokeType::curved, PathStrokeType::rounded));

// --- Línea indicadora (desde centro hasta borde interior) ---
auto thumbR = radius - 11.0f;
Point<float> thumbPt(center.x + thumbR * std::sin(angle),
    center.y - thumbR * std::cos(angle));
g.setColour(Colour(210, 210, 210));
g.drawLine(Line<float>(center, thumbPt), 2.0f);

// --- Punto central ---
g.setColour(Colour(180, 180, 180));
g.fillEllipse(Rectangle<float>(5.0f, 5.0f).withCentre(center));

// --- Valor numérico (centrado en el knob) ---
if (auto* rsw = dynamic_cast<RotarySliderWithLabels*>(&slider))
{
    g.setColour(Colour(200, 200, 200));
    g.setFont(FontOptions(10.5f));
    g.drawFittedText(rsw->getDisplayString(),
        bounds.withSizeKeepingCentre(radius * 1.4f, radius * 0.6f).toNearestInt(),
        Justification::centred, 1);

    // --- Nombre del parámetro debajo del knob ---
    g.setColour(Colour(120, 120, 130));
    g.setFont(FontOptions(10.0f));
    auto labelBounds = Rectangle<int>(x, y + height - labelH, width, labelH);
    g.drawFittedText(rsw->param->getName(24), labelBounds, Justification::centred, 1);
}
}
```

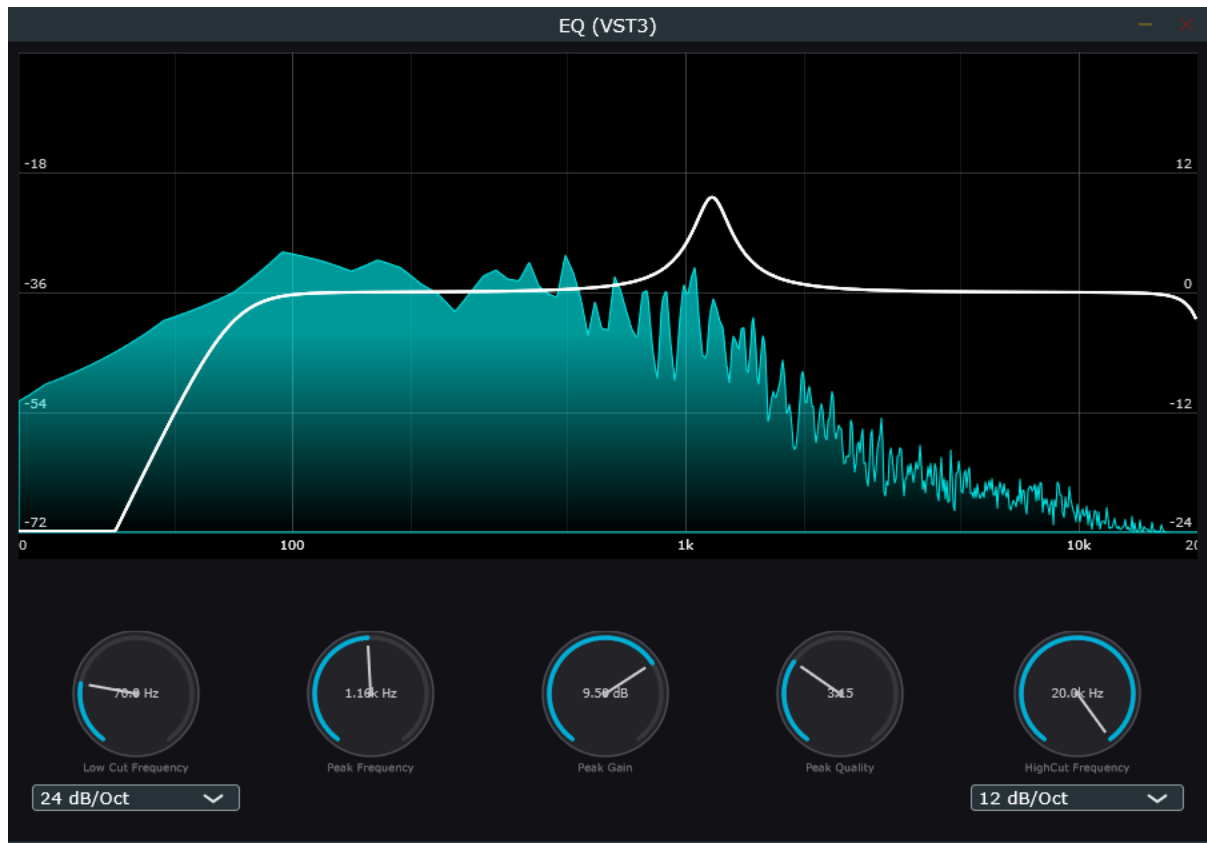
Estética y Vinculación:

- Componentes Personalizados: Se ha creado la clase `RotarySliderWithLabels`, la cual hereda de `juce::Slider`. Mediante una sobreescripción de `juce::LookAndFeel_V4`, se ha redefinido el método `drawRotarySlider` para dibujar a medida el dial del potenciómetro, el indicador de posición y las etiquetas numéricas de los rangos.
- Uso de Attachments: Se han instanciado los objetos `SliderAttachment` y `ComboBoxAttachment`. Estos garantizan una sincronización atómica entre lo que el usuario ve en la interfaz y los valores reales del árbol de estado de parámetros (APVTS).
- Gestión de Layout Proporcional: En el método `resized()` del `PluginEditor`, se dividen dinámicamente las áreas utilizando `removeFromTop` y `removeFromLeft`, asegurando que la interfaz se adapte elegantemente sin codificar valores absolutos y fijos (hardcoding).

El uso de las clases de Attachment proporcionadas por JUCE previene problemas clásicos de concurrencia y retroalimentación infinita de valores al girar un potenciómetro. Además, la segregación del dibujo dentro de un LookAndFeel personalizado mantiene la clase del componente visual limpia, promoviendo la reutilización del código de diseño en futuros plugins.

15. Renderizado Visual del Analizador de Espectro (FFT)

Con la infraestructura de datos (FIFOs) y el motor lógico del analizador ya implementados en el procesador, en esta fase se ha completado la representación gráfica del espectro de audio sobre el componente de la interfaz.



Normalización y Trazado:

- Rango Dinámico Constante: En PluginEditor.h se han definido las constantes `NOISE_FLOOR = -72.0f` y `MAX_DB = 0.0f` dentro de `SpectrumAnalyzer`. Esto establece los límites visuales del gráfico, asegurando que las frecuencias muy bajas (ruido de fondo) no se dibujen y que la señal más fuerte no sobrepase la ventana.
- Mapeo al Eje Y: Durante el método `paint`, los valores de magnitud de la FFT (previamente normalizados entre 0 y 1) se transforman en coordenadas verticales. El valor de 0 (o menor a `NOISE_FLOOR`) se dibuja en la base del componente, mientras que los picos se elevan proporcionalmente.

El renderizado visual de la FFT unifica la retroalimentación auditiva y visual. Al limitar el rango dinámico con `NOISE_FLOOR` y procesar el enventanado (Hann window), se evita mostrar "basura espectral", ofreciendo al usuario una lectura limpia y profesional del contenido frecuencial de su señal.

16. Control Interactivo sobre la Curva (Gestión del Ratón)

Se ha transformado el ResponseCurveComponent de ser un simple visor estático a una superficie de control activa, permitiendo al usuario modificar la ecualización haciendo clic y arrastrando directamente sobre el gráfico.

```
static float mapXToFreq(float x, float leftX, float width)
{
    static const float logMin = std::log10(20.0f);
    static const float logMax = std::log10(20000.0f);
    float normalizedX = (x - leftX) / width;
    return std::pow(10.0f, logMin + normalizedX * (logMax - logMin));
}

static float mapFreqToX(float freq, float leftX, float width)
{
    // Eje X: log10(20..20000) → 0..width
    static const float logMin = std::log10(20.0f);
    static const float logMax = std::log10(20000.0f);
    float logFreq = std::log10(juce::jlimit(20.0f, 20000.0f, freq));
    return leftX + width * (logFreq - logMin) / (logMax - logMin);
}
```

Matemáticas de Interacción:

- Conversión Inversa (mapXToFreq): Se ha añadido una función de utilidad matemática que realiza el proceso inverso a dibujar. Toma la coordenada X del píxel donde el usuario ha hecho clic y la convierte de nuevo a una frecuencia en Hercios (Hz) dentro de la escala logarítmica de 20 Hz a 20 kHz.
- Actualización del APVTS: Al detectar eventos de ratón (mouseDown, mouseDrag), el editor calcula la nueva frecuencia basándose en la posición del cursor y actualiza instantáneamente el valor del parámetro en el AudioProcessorValueTreeState.

Integrar el control del ratón directamente en el gráfico requiere una traducción constante entre el dominio visual (píxeles lineales) y el dominio de audio (frecuencias logarítmicas). Esta interactividad mejora drásticamente el flujo de trabajo del usuario (UX), permitiéndole "tocar" el sonido en lugar de depender únicamente de los potenciómetros inferiores.

17. Refinamiento del Layout y UX (Experiencia de Usuario)

Se han realizado ajustes significativos en el método `resized()` del `PluginEditor` para asegurar que los controles se comporten de manera lógica y estética cuando el usuario redimensiona la ventana del plugin.

```
void EQAudioProcessorEditor::resized()
{
    auto area = getLocalBounds().reduced(8);

    // 1. El visualizador ocupa el 65% superior
    auto displayArea = area.removeFromTop(juce::roundToInt(area.getHeight() * 0.65f));
    responseCurveComponent.setBounds(displayArea);

    // 2. Separación extra entre la gráfica y los controles
    area.removeFromTop(15);

    // 3. Forzamos una altura máxima para las ruedas para que no se vean gigantes
    auto controlsArea = area;
    int maxSliderHeight = 110;
    if (controlsArea.getHeight() > maxSliderHeight) {
        // Centramos el rectángulo de controles si nos sobra espacio vertical
        controlsArea = controlsArea.withSizeKeepingCentre(controlsArea.getWidth(), maxSliderHeight);
    }

    int sliderW = controlsArea.getWidth() / 5;

    // Distribuimos los sliders
    lowCutFreqSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakFreqSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakGainSlider.setBounds(controlsArea.removeFromLeft(sliderW));
    peakQualitySlider.setBounds(controlsArea.removeFromLeft(sliderW));
    highCutFreqSlider.setBounds(controlsArea);

    // 4. Ubicación de los comboboxes de Slope
    // Los colocamos justo debajo de sus respectivos controles de LowCut y HighCut
    int comboH = 20;

    // LowCut ComboBox debajo del LowCut Freq
    auto lowCutBounds = lowCutFreqSlider.getBounds();
    lowCutSlopeBox.setBounds(lowCutBounds.getX() + 10, lowCutBounds.getBottom() + 5, lowCutBounds.getWidth() - 20, comboH);

    // HighCut ComboBox debajo del HighCut Freq
    auto highCutBounds = highCutFreqSlider.getBounds();
    highCutSlopeBox.setBounds(highCutBounds.getX() + 10, highCutBounds.getBottom() + 5, highCutBounds.getWidth() - 20, comboH);
}
```

Control de Proporciones:

- Límite de Escala (`maxSliderHeight`): Se ha introducido una restricción de 110 píxeles para la altura de los controles rotatorios (`controlsArea.withSizeKeepingCentre`). Esto previene el comportamiento indeseado donde los potenciómetros se volvían gigantes y pixelados al maximizar la ventana en monitores grandes.
- Agrupación Semántica: Los selectores de pendiente (`lowCutSlopeBox` y `highCutSlopeBox`) ahora se posicionan dinámicamente de forma relativa justo debajo de sus respectivos potenciómetros de frecuencia. En lugar de ocupar una porción del layout general, se calculan utilizando `getBottom()` del área del slider, manteniéndolos visualmente agrupados con el control al que afectan.

Un diseño verdaderamente responsive en C++ no solo consiste en dividir porcentajes de pantalla, sino en definir límites lógicos (`minWidth`, `maxHeight`) para proteger la integridad visual. Agrupar espacialmente los controles dependientes refuerza el modelo mental del usuario sobre la ruta de la señal.

18. Gestión Rápida de Estado: Botón "Default" (Reinicio de Parámetros)

Para mejorar la experiencia del usuario (UX) y agilizar el flujo de trabajo, se ha introducido un botón global de "Default". Este botón permite devolver instantáneamente todos los filtros del ecualizador a su posición neutra, ahorrando al usuario la tarea de resetear los potenciómetros uno por uno.

```
void EQAudioProcessorEditor::resetToDefaults()
{
    // Resetear los parámetros flotantes a sus valores por defecto
    audioProcessor.apvts.getParameter("LowCut Frequency")->setValueNotifyingHost(0.0f); // 20 Hz
    audioProcessor.apvts.getParameter("HighCut Frequency")->setValueNotifyingHost(1.0f); // 20000 Hz
    audioProcessor.apvts.getParameter("Peak Frequency")->setValueNotifyingHost(audioProcessor.apvts.getParameter("Peak Frequency")->getDefaultValue());
    audioProcessor.apvts.getParameter("Peak Gain")->setValueNotifyingHost(0.5f); // 0 dB (centro del rango)
    audioProcessor.apvts.getParameter("Peak Quality")->setValueNotifyingHost(audioProcessor.apvts.getParameter("Peak Quality")->getDefaultValue());

    // Resetear los slopes a 12 dB/Oct (índice 0)
    audioProcessor.apvts.getParameter("LowCut Slope")->setValueNotifyingHost(0.0f);
    audioProcessor.apvts.getParameter("HighCut Slope")->setValueNotifyingHost(0.0f);
}
```

Lógica y Sincronización Segura:

- Implementación de Interfaz: Se ha añadido una instancia de `juce::TextButton` en `PluginEditor.h`. En el constructor del editor, se le asigna una función lambda al evento `onClick`, la cual dispara el método `resetToDefaults()` cuando el usuario hace clic.
- Integración en el Layout: En el método `resized()`, se ha reservado una franja superior dedicada de 20 píxeles (`removeFromTop(20)`) para alojar utilidades, asignando el botón a la esquina superior izquierda del área de controles.
- Sincronización Atómica (Notificación al Host): El aspecto técnico más crítico de esta implementación es el uso de `setValueNotifyingHost(...)` sobre los parámetros del APVTS. En lugar de cambiar visualmente los sliders de la interfaz, el código modifica directamente el árbol de estado subyacente.

Al utilizar `setValueNotifyingHost()`, se garantiza una "cascada de eventos" perfecta y segura frente a la concurrencia de hilos (thread-safe). Cuando se pulsa el botón, el APVTS actualiza su valor interno, notifica inmediatamente a las clases de tipo `Attachment` (lo que hace que los potenciómetros visuales se muevan solos a su posición cero), notifica al procesador de audio para que recalcule los coeficientes matemáticos de los filtros, y avisa al Host (el DAW) para que registre el cambio a nivel de automatización. Todo ello en un solo paso.

19. Expansión de la Cadena DSP: Ecualizador Paramétrico de 5 Bandas

Para ofrecer mayor versatilidad y control sobre el espectro, se ha ampliado la arquitectura del motor de audio, pasando de un diseño básico (LowCut, Peak, HighCut) a un ecualizador paramétrico completo con tres bandas centrales independientes.

```
// --- Tipos del procesador de filtros ---
using Filter = juce::dsp::IIR::Filter<float>;
using CutFilter = juce::dsp::ProcessorChain<Filter, Filter, Filter, Filter>;
using MonoChain = juce::dsp::ProcessorChain<CutFilter, Filter, Filter, Filter, CutFilter>;
MonoChain leftChain, rightChain;

private:

    enum ChainPositions { LowCut, Peak, Peak2, Peak3, HighCut };

    // --- Actualización de filtros ---
    void updatePeakFilter      (const ChainSettings& s);
    void updatePeak2Filter     (const ChainSettings& s);
    void updatePeak3Filter     (const ChainSettings& s);
    void updateLowCutFilters   (const ChainSettings& s);
    void updateHighCutFilters  (const ChainSettings& s);
    void updateFilters();
```

Ampliación de la Arquitectura:

- Modificación de MonoChain: El alias de la cadena principal se ha redefinido para alojar cinco eslabones: `juce::dsp::ProcessorChain<CutFilter, Filter, Filter, Filter, CutFilter>`.
- Nuevos Parámetros en APVTS: Se han añadido seis nuevos parámetros flotantes al árbol de estado correspondientes a la frecuencia, ganancia y calidad (Q) de Peak2 y Peak3.
- Procesamiento de Coeficientes: Se han replicado y adaptado las funciones de cálculo (`updatePeak2Filter` y `updatePeak3Filter`) para que el procesador alimente independientemente los nodos 1, 2 y 3 de la cadena con sus respectivos coeficientes.

Escalar el número de bandas ha sido un proceso limpio y predecible. Al haber invertido previamente en una arquitectura basada en `juce::dsp::ProcessorChain`, añadir nuevos filtros no requiere complejas matemáticas de ruteo; basta con definir el eslabón en la cadena, registrar el parámetro en el APVTS y enviarle los coeficientes calculados, manteniendo el hilo de audio altamente optimizado.

20. Sistema de Gestión de Estados: Bypass por Banda

Se ha implementado una funcionalidad crucial para el flujo de trabajo en mezcla: la capacidad de encender y apagar (hacer bypass) cada filtro de forma individual para escuchar comparativas "A/B" sin perder la configuración de los parámetros.

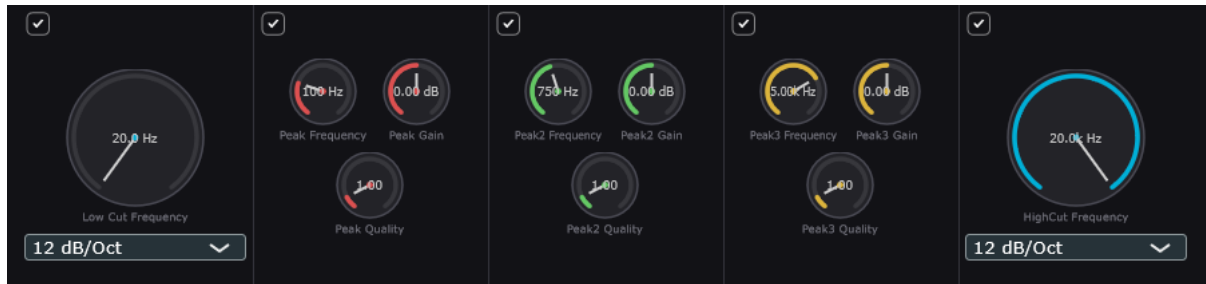
```
template<typename ChainType, typename CoefficientType>
void updateCutFilter(ChainType& chain,
                    const CoefficientType& coefficients,
                    const Slope& slope)
{
    chain.template setBypassed<0>(true);
    chain.template setBypassed<1>(true);
    chain.template setBypassed<2>(true);
    chain.template setBypassed<3>(true);
}
```

Lógica de Desactivación Segura:

- Parámetros Booleanos: Se han añadido parámetros de tipo `juce::AudioParameterBool` al APVTS para representar el estado de encendido/apagado de cada banda (ej. "Peak Bypass").
- Integración en el DSP: La función plantilla `update` ha sido modificada para recibir el estado de bypass desde `ChainSettings`. Utiliza el método nativo `chain.template setBypassed<Index>(bypassed)`, lo que le dice al motor DSP que la señal debe saltarse ese nodo matemático de forma transparente, ahorrando además ciclos de CPU.

21. Evolución de la Interfaz Gráfica: Layout Multibanda y Retroalimentación Visual

El PluginEditor ha sufrido una importante reestructuración para acomodar los nuevos controles físicos y reflejar visualmente el estado de activación de cada banda.



Diseño y Usabilidad (UX/UI):

- Reorganización del Layout: El espacio inferior de controles se divide ahora dinámicamente en 5 columnas ($\text{sliderW} = \text{controlsArea.getWidth()} / 5$).
- Botones de Activación (`JUCE::ToggleButton`): Se ha añadido un botón tipo interruptor en la parte superior de cada columna de controles. Mediante `ButtonAttachment`, estos se comunican bidireccionalmente con los booleanos de bypass del APVTS.
- Feedback Visual de Bypass: La clase `RotarySliderWithLabels` detecta ahora si su filtro está en bypass. Si es así, reduce la opacidad/color de los controles, indicando al usuario visualmente que ese filtro no está afectando a la señal de audio en ese momento.
-

22. Interactividad Avanzada en el Visor de Curvas

El componente `ResponseCurveComponent` se ha enriquecido para representar y permitir la interacción con las nuevas bandas paramétricas de forma intuitiva.

Código de Colores e Interacción Multizona:

- Diferenciación Visual: Para evitar confusiones visuales, se han definido constantes de color (`PEAK1_COLOR`, `PEAK2_COLOR`, `PEAK3_COLOR`). Cada nodo de pico se dibuja con su respectivo color en la pantalla.
- Detección de Proximidad: Durante los eventos de `mouseDown` y `mouseDrag`, el algoritmo ahora calcula la distancia del cursor respecto a la posición (X, Y) de los tres nodos paramétricos. El sistema "engancha" y modifica automáticamente el filtro más cercano al ratón, permitiendo una edición gráfica fluida sin necesidad de seleccionar previamente qué banda se desea ajustar.

23. Resultado del Release 1:

Introducción: Release 2:

El Release 2 representa un avance significativo en la interactividad y usabilidad del ecualizador paramétrico. Mientras que el Release 1 estableció una arquitectura sólida de procesamiento DSP (LowCut, Peak, HighCut) con visualización de espectro y curva de respuesta, el Release 2 se enfoca en transformar la curva de respuesta en una superficie de control activa e intuitiva.

24. Interactividad Gráfica Avanzada: Edición Directa sobre la Curva de Respuesta:

Este es el cambio más significativo del Release 2. El ResponseCurveComponent ha sido transformado de un simple visor estático a una superficie de control activa que permite al usuario editar todos los parámetros de EQ de forma intuitiva arrastrando directamente sobre el gráfico de respuesta de frecuencia.

Detección de Peaks y Hit-Testing

El sistema implementa un algoritmo de detección de proximidad que continuamente calcula la distancia entre la posición del cursor y cada uno de los tres puntos de picos (Peak, Peak2, Peak3). Cuando el usuario presiona el botón del ratón dentro de un radio de selección (típicamente 30 píxeles), el algoritmo identifica el pico más cercano como 'activo' y entra en modo de edición.

Este enfoque hit-test proporciona un área de captura forgiving que hace la edición fluida y cómoda, sin requerir precisión milimétrica. Los tres picos están diferenciados por color (Rojo, Verde, Amarillo) lo que permite identificación visual inmediata de cuál se está editando.

Mapeo Logarítmico Frecuencia/Píxeles

La lógica crítica del sistema son dos funciones de mapeo que traducen entre el dominio visual (píxeles lineales en la pantalla) y el dominio de audio (frecuencias logarítmicas de 20 Hz a 20 kHz):

- mapFreqToX(float freq): Convierte una frecuencia en Hercios a una coordenada X en píxeles
- mapXToFreq(float x): Realiza la conversión inversa, transformando píxeles a frecuencias

Ambas funciones utilizan aritmética logarítmica (log base 10) para mantener la relación matemáticamente correcta entre la visualización en pantalla y las frecuencias reales. Esto asegura que, cuando el usuario arrastra el ratón una distancia igual en píxeles, el cambio de frecuencia es consistente independientemente de si está editando en los graves (20-200 Hz) o en los agudos (5-20 kHz).

```
// Ejemplo simplificado del mapeo logarítmico
float mapFreqToX(float freq, float leftX, float width) {
    const float logMin = log10(20.0f);
    const float logMax = log10(20000.0f);
    float normalized = (log10(freq) - logMin) / (logMax - logMin);
    return leftX + normalized * width;
}

float mapXToFreq(float x, float leftX, float width) {
    const float logMin = log10(20.0f);
    const float logMax = log10(20000.0f);
    float normalized = (x - leftX) / width;
    return std::pow(10.0f, logMin + normalized * (logMax - logMin));
}
```

Edición de Parámetros: Freq, Gain y Q

Durante el arrastre (mouseDrag), el sistema actualiza tres parámetros independientes:

- Frecuencia (X): El movimiento horizontal del ratón se traduce directamente a cambios de frecuencia. La escala logarítmica asegura que el usuario sienta igual 'control' en todo el espectro.
- Ganancia (Y): El movimiento vertical se mapea a cambios de ganancia en decibelios. Hacia arriba = boost, hacia abajo = corte. La sensibilidad es constante en píxeles/dB.
- Factor Q (Rueda): La rueda del ratón ajusta el factor de calidad (Q) multiplicativamente. Con Ctrl presionado, la sensibilidad se multiplica por 3 para cambios rápidos. Sin Ctrl, es más preciso.

Propagación Inmediata al APVTS

Cada cambio de parámetro se propaga inmediatamente al AudioProcessorValueTreeState mediante setValueNotifyingHost(). Esta función desencadena una cascada de eventos sincronizados:

- El APVTS actualiza su valor interno de forma atómica.
- Las clases SliderAttachment detectan el cambio y actualizan visualmente los knobs inferiores.
- El procesador de audio (backend) es notificado para recalcular los coeficientes de los filtros.
- El host DAW (si está en modo plugin) registra el cambio para automatización.
- La curva de respuesta se redibuja automáticamente reflejando el nuevo estado.

Todo ocurre sin latencia perceptible, dando al usuario la sensación de control directo sobre el sonido. Es una interactividad de retroalimentación inmediata que es fundamental para la creatividad en tiempo real.

Gestión de Gestos y Automatización

Para que el host DAW registre correctamente los cambios durante un arrastre, se implementan dos métodos críticos:

- beginGestureForPeak(int peak): Notifica al host que se inicia una edición de ese pico
- endGestureForPeak(int peak): Notifica que la edición ha terminado

Estos métodos permiten que sistemas de automatización avanzados registren toda la secuencia de cambios como una sola 'frase' coherente, en lugar de como miles de eventos individuales. En un DAW moderno, esto se refleja como una curva de automatización suave que rastrea exactamente cómo el usuario mueve el ratón durante el arrastre.



25. Sistema de Temas Visuales: Skins

Para mejorar la versatilidad y permitir que el plugin se adapte a diferentes preferencias estéticas y contextos de trabajo, se ha implementado un sistema modular de temas (skins) que permite cambiar la apariencia visual del plugin sin afectar ningún aspecto del procesamiento de audio.

Arquitectura del Sistema de Skins

La solución está basada en una estructura C++ denominada 'Skin' que contiene alrededor de 30 colores, cada uno asignado a un elemento visual específico del plugin. Al cambiar de tema, simplemente se apunta a una estructura 'Skin' diferente, y todos los componentes de la GUI automáticamente usan los nuevos colores en su próximo ciclo de paint().

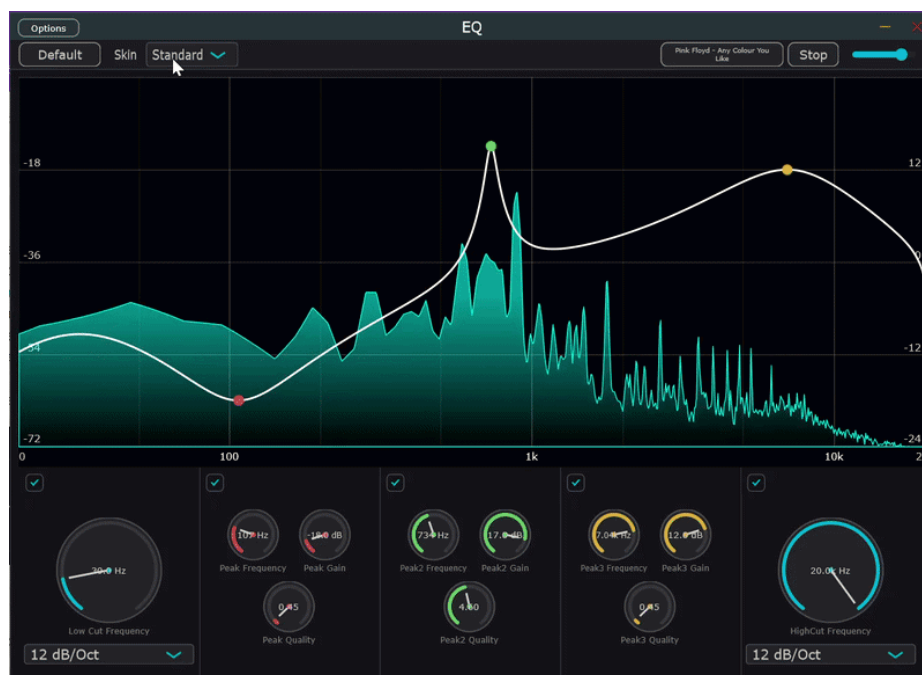
Esta arquitectura es extremadamente eficiente porque no requiere ningún procesamiento costoso: es solo un cambio de puntero. No hay necesidad de recalcular geometrías, redimensionar componentes, o realizar ninguna operación de GPU intensiva.

Tres Temas Predefinidos

- Standard (Studio Dark): El tema por defecto. Utiliza una paleta oscura (gris oscuro casi negro como fondo) con acentos cian brillante para controles. Está optimizado para estudios con iluminación controlada y proporciona un aspecto profesional moderno.
- Dark (Vintage Analog): Un tema que evoca ecualizadores analógicos vintage con acabado de madera oscura, detalles en latón, y tonos cálidos (óxido, mostaza). El gráfico utiliza colores cálidos (cobre y crema) que simulan la estética de instrumentos clásicos de los años 70-80.
- Light: Un tema claro ideal para ambientes muy iluminados o para usuarios que prefieren fondos blancos. Utiliza tonos pastel restringidos y colores de material design vivos para los picos. Proporciona alto contraste para legibilidad en cualquier condición de iluminación.

Selección de Tema desde la Interfaz

Un ComboBox situado en la franja superior derecha del plugin permite al usuario seleccionar entre los tres temas. El cambio de tema es instantáneo y no causa ningún cambio en el estado de los parámetros de audio o en la automatización. La selección se preserva en la sesión hasta que el usuario seleccione un tema diferente.



26. Reproductor de Archivos Integrado (File Player):

El Release 2 incluye un sistema completo de reproducción de archivos de audio que funciona exclusivamente en el modo standalone del plugin. Esta característica permite al usuario cargar y reproducir archivos de audio a través del ecualizador para evaluar cómo los ajustes de EQ afectan al sonido del material de origen en tiempo real.

Formatos de Audio Soportados

El reproductor soporta todos los formatos que JUCE puede decodificar de forma nativa:

- WAV (sin compresión)
- AIFF (sin compresión)
- MP3 (comprimido)
- FLAC (comprimido sin pérdida)
- OGG Vorbis (comprimido)

La auto-detección de formato es automática basándose en la extensión del archivo y su cabecera, por lo que el usuario simplemente selecciona un archivo y el reproductor maneja el decodificador apropiado.

Arquitectura Thread-Safe del File Player

El File Player opera sobre tres componentes principales que están sincronizados mediante un `juce::CriticalSection` (`playerLock`):

- `juce::AudioTransportSource`: Gestor de transporte que mantiene la posición de reproducción, velocidad de playback y estado (`playing/paused/stopped`).
- `juce::AudioFormatReaderSource`: Decodificador que extrae muestras de audio del archivo. Maneja tanto archivos sin compresión como comprimidos de forma transparente.
- `juce::AudioFormatManager`: Registrador de decodificadores que permite que JUCE auto-detecte qué decodificador usar basándose en la extensión del archivo.

Integración en el ProcessBlock

Dentro del método `processBlock`, se utiliza un `juce::ScopedTryLock` para intentar adquirir el lock sin bloquear nunca el hilo de audio. Si el lock se adquiere exitosamente y el reproductor está tocando, se obtiene el siguiente bloque de audio desde el archivo, se aplica la ganancia del volumen, y el buffer resultante reemplaza completamente la entrada del procesador. El audio del archivo fluye a través de la cadena completa de filtros DSP.

Esta arquitectura garantiza que aunque el usuario interactúe con los botones 'Load', 'Play', 'Stop' desde el hilo de la interfaz gráfica, el hilo de audio nunca quedará bloqueado esperando, lo que previene chasquidos (glitches) o interrupciones en el procesamiento.

Controles de la Interfaz

Tres botones en la franja superior controlan el reproductor:

- Input Button: Abre un file chooser que permite seleccionar un archivo de audio. Una vez cargado, el nombre del archivo aparece en el botón.
- Play/Stop Button: Alterna entre reproducción y pausa del archivo cargado. El etiquetado del botón cambia dinámicamente según el estado.
- Volume Slider: Un slider que aparece solo cuando hay un archivo cargado, permitiendo controlar el volumen de mezcla del reproductor (0..1).

